

Skyfield: [Home](#) • [Table of Contents](#) • [Changelog](#) • [API Reference](#)

API Reference¶

Quick links to the sections below:

- [Version](#)
- [Opening files](#)
- [Time scales](#)
- [Time objects](#)
- [Time utilities](#)
- [Vector functions](#)
- [Planetary ephemerides](#)
- [Planetary magnitudes](#)
- [Planetary reference frames](#)
- [Almanac](#)
- [Geographic locations](#)
- [Kepler orbits](#)
- [Kepler orbit data](#)
- [Earth satellites](#)
- [Stars and other distant objects](#)
- [Astronomical positions](#)
- [Reference frames](#)
- [Constellations](#)
- [Searching](#)
- [Osculating orbital elements](#)
- [Units](#)
- [Trigonometry](#)

Version¶

Skyfield offers a tuple `skyfield.VERSION` that lets your code determine the installed version of Skyfield.

```
import skyfield
print(skyfield.VERSION)
```

See [Checking your Skyfield version](#).

Opening files¶

```
# File you already have.
```

```
from skyfield.api import load_file
planets = load_file('~Downloads/de405.bsp')
```

`load_file`(path) Open a file on your local drive, using its extension to guess its type.

```
# File you want Skyfield to download automatically.

from skyfield.api import load
ts = load.timescale()
planets = load('de405.bsp')
```

<code>Loader</code> (directory[, verbose, expire])	A tool for downloading and opening astronomical data files.
<code>Loader.build_url</code> (filename)	Return the URL Skyfield will try downloading for a given filename.
<code>Loader.days_old</code> (filename)	Return how recently <code>filename</code> was modified, measured in days.
<code>Loader.download</code> (url[, filename, backup])	Download a file, even if it's already on disk; return its path.
<code>Loader.path_to</code> (filename)	Return the path to <code>filename</code> in this loader's directory.
<code>Loader.timescale</code> ([delta_t, builtin])	Return a <code>Timescale</code> built using official Earth rotation data.
<code>Loader.tle_file</code> (url[, reload, filename, ts, ...])	Load and parse a TLE file, returning a list of Earth satellites.

Time scales¶

A script will typically start by building a single Skyfield `Timescale` to use for all date and time conversions:

```
from skyfield import api
ts = api.load.timescale()
```

Its methods are:

<code>Timescale.now</code> ()	Return the current date and time as a <code>Time</code> object.
<code>Timescale.from_datetime</code> (datetime)	Return a <code>Time</code> for a Python <code>datetime</code> .
<code>Timescale.from_datetimes</code> (datetime_list)	Return a <code>Time</code> for a list of Python <code>datetime</code> objects.
<code>Timescale.utc</code> (year[, month, day, hour, ...])	Build a <code>Time</code> from a UTC Calendar date .

<code>Timescale.tai</code> ([year, month, day, hour, ...])	Build a <code>Time</code> from an International Atomic Time <code>Calendar date</code> .
<code>Timescale.tai_jd</code> (jd[, fraction])	Build a <code>Time</code> from an International Atomic Time Julian date.
<code>Timescale.tt</code> ([year, month, day, hour, ...])	Build a <code>Time</code> from a Terrestrial Time <code>Calendar date</code> .
<code>Timescale.tt_jd</code> (jd[, fraction])	Build a <code>Time</code> from a Terrestrial Time Julian date.
<code>Timescale.J</code> (year)	Build a <code>Time</code> from a Terrestrial Time Julian year or array.
<code>Timescale.tdb</code> ([year, month, day, hour, ...])	Build a <code>Time</code> from a Barycentric Dynamical Time <code>Calendar date</code> .
<code>Timescale.tdb_jd</code> (jd[, fraction])	Build a <code>Time</code> from a Barycentric Dynamical Time Julian date.
<code>Timescale.ut1</code> ([year, month, day, hour, ...])	Build a <code>Time</code> from a UT1 Universal Time <code>Calendar date</code> .
<code>Timescale.ut1_jd</code> (jd)	Build a <code>Time</code> from a UT1 Universal Time Julian date.
<code>Timescale.from_astropy</code> (t)	Build a Skyfield <code>Time</code> from an AstroPy time object.
<code>Timescale.linspace</code> (t0, t1[, num])	Return <code>num</code> times spaced uniformly between <code>t0</code> to <code>t1</code> .

Time objects¶

The `Time` class is Skyfield’s way of representing either a single time, or a whole array of times. The same time can be represented in several different time scales.

<code>t.tai</code>	International Atomic Time (TAI) as a Julian date.
<code>t.tt</code>	Terrestrial Time (TT) as a Julian date.
<code>t.J</code>	Terrestrial Time (TT) as floating point Julian years.
<code>t.tdb</code>	Barycentric Dynamical Time (TDB) as a Julian date.
<code>t.ut1</code>	Universal Time (UT1) as a Julian date.

A couple of offsets between time scales are also available.

<code>t.delta_t</code>	Difference TT - UT1 in seconds.
<code>t.dut1</code>	Difference UT1 - UTC in seconds.

Other time scales and conversions are available through its methods.

<code>Time.utc_jpl()</code>	Convert to a string like <code>A.D.2014-Jan-18 01:35:37.5000 UTC..</code>
<code>Time.utc_iso([delimiter, places])</code>	Convert to an ISO 8601 string like <code>2014-01-18T01:35:38Z</code> in UTC.
<code>Time.utc_strftime([format])</code>	Format the UTC time using a Python datetime formatting string.
<code>Time.utc_datetime()</code>	Convert to a Python <code>datetime</code> in UTC.
<code>Time.utc_datetime_and_leap_second()</code>	Convert to a Python <code>datetime</code> in UTC, plus a leap second value.
<code>Time.astimezone(tz)</code>	Convert to a Python <code>datetime</code> in a particular timezone <code>tz</code> .
<code>Time.astimezone_and_leap_second(tz)</code>	Convert to a Python <code>datetime</code> and leap second in a timezone.
<code>Time.toordinal()</code>	Return the proleptic Gregorian ordinal of the UTC date.
<code>Time.tai_calendar()</code>	TAI as a (year, month, day, hour, minute, second) Calendar date .
<code>Time.tt_calendar()</code>	TT as a (year, month, day, hour, minute, second) Calendar date .
<code>Time.tdb_calendar()</code>	TDB as a (year, month, day, hour, minute, second) Calendar date .
<code>Time.ut1_calendar()</code>	UT1 as a (year, month, day, hour, minute, second) Calendar date .
<code>Time.tai_strftime([format])</code>	Format TAI with a datetime <code>strftime()</code> format string.
<code>Time.tt_strftime([format])</code>	Format TT with a datetime <code>strftime()</code> format string.
<code>Time.tdb_strftime([format])</code>	Format TDB with a datetime <code>strftime()</code> format string.
<code>Time.ut1_strftime([format])</code>	Format UT1 with a datetime <code>strftime()</code> format string.
<code>Time.M</code>	3×3 rotation matrix: ICRS $\rightarrow$ equinox of this date.
<code>Time.MT</code>	3×3 rotation matrix: equinox of this date $\rightarrow$ ICRS.
<code>Time.gmst</code>	Greenwich Mean Sidereal Time (GMST) in hours.
<code>Time.gast</code>	Greenwich Apparent Sidereal Time (GAST) in hours.
<code>Time.nutation_matrix()</code>	Compute the 3×3 nutation matrix N for this date.
<code>Time.precession_matrix()</code>	Compute the 3×3 precession matrix P for this date.
<code>Time.to_astropy()</code>	Return an <code>AstroPy</code> object representing this time.

Time utilities¶

`compute_calendar_date`(jd_integer[, ...]) Convert Julian day `jd_integer` into a calendar (year, month, day).

Vector functions¶

The common API shared by planets, Earth locations, and Earth satellites.

`VectorFunction` Given a time, computes a corresponding position.

`VectorFunction.at`(t) At time `t`, compute the target's position relative to the center.

Either adding two vector functions `v1 + v2` or subtracting them `v1 - v2` produces a new function of time that, when invoked with `.at(t)`, returns the sum or difference of the vectors returned by the two functions.

Planetary ephemerides¶

By downloading a `SpiceKernel` file, Skyfield users can build vector functions predicting the positions of the Moon, Sun, and planets. See [Planets and their moons: JPL ephemeris files](#).

`SpiceKernel`(path) Ephemeris file in NASA .bsp format.

`SpiceKernel.close`() Close this ephemeris file.

`SpiceKernel.comments`() Return the comments string of this kernel.

`SpiceKernel.names`() Return all target names that are valid with this kernel.

`SpiceKernel.decode`(name) Translate a target name into its integer code.

Kernels also support lookup using the Python `kernel['Mars']` syntax, in which case they return a function of time that returns vectors from the Solar System barycenter to the named body.

Planetary magnitudes¶

`skyfield.magnitudelib.planetary_magnitude(position)¶`

Given the position of a planet, return its visual magnitude.

```
>>> from skyfield.api import load
>>> from skyfield.magnitudelib import planetary_magnitude
>>> ts = load.timescale()
>>> t = ts.utc(2020, 7, 31)
>>> eph = load('de421.bsp')
>>> astrometric = eph['earth'].at(t).observe(eph['jupiter barycenter'])
```

```
>>> print('%0.2f' % planetary_magnitude(astrometric))
-2.73
```

The formulae are from [Mallama and Hilton “Computing Apparent Planetary Magnitude for the Astronomical Almanac” \(2018\)](#). Two of the formulae have inherent limits:

- Saturn’s magnitude is unknown and the function will return `nan` (the floating-point value “Not a Number”) if the “illumination phase angle” — the angle of the vertex observer-Saturn-Sun — exceeds 6.5° .
- Neptune’s magnitude is unknown and will return `nan` if the illumination phase angle exceeds 1.9° and the position’s date is before the year 2000.

And one formula is not fully implemented (though contributions are welcome!):

- Skyfield does not compute which features on Mars are facing the observer, which can introduce an error of ± 0.06 magnitude.

Planetary reference frames¶

[PlanetaryConstants](#) Planetary constants manager.

[Frame](#) Planetary constants frame, for building rotation matrices.

Almanac¶

Routines to search for events like sunrise, sunset, and Moon phase.

find_risings (observer, target, start_time, ...)	Return the times at which a target rises above the eastern horizon.
find_settings (observer, target, start_time, ...)	Return the times at which a target sets below the western horizon.
find_transits (observer, target, start_time, ...)	Return the times at which a target transits across the meridian.
seasons (ephemeris)	Build a function of time that returns the quarter of the year.
moon_phase (ephemeris, t)	Return the Moon phase 0° – 360° at time <code>t</code> , where 180° is Full Moon.
moon_phases (ephemeris)	Build a function of time that returns the moon phase 0 through 3.
moon_nodes (ephemeris)	Build a function of time that identifies lunar nodes.

<code>oppositions_conjunctions</code> (ephemeris, target)	Build a function to find oppositions and conjunctions with the Sun.
<code>meridian_transits</code> (ephemeris, target, topos)	Build a function of time for finding when a body transits the meridian.
<code>sunrise_sunset</code> (ephemeris, topos)	Build a function of time that returns whether the Sun is up.
<code>dark_twilight_day</code> (ephemeris, topos)	Build a function of time returning whether it is dark, twilight, or day.
<code>risings_and_settings</code> (ephemeris, target, topos)	Build a function of time that returns whether a body is up.
<code>lunar_eclipses</code> (start_time, end_time, eph)	Return the lunar eclipses between <code>start_time</code> and <code>end_time</code> .

Geographic locations ¶

Skyfield supports two Earth datums for translating between latitude/longitude and Cartesian coordinates. They each use a slightly different estimate of the Earth's oblateness. The most popular is WGS84, which is used by the world's GPS devices:

- `wgs84`
- `iers2010`

Each datum offers a method for taking a latitude and longitude and returning a `GeographicPosition` that knows its position in space:

`Geoid.latlon` (latitude_degrees, longitude_degrees) Return a `GeographicPosition` for a given latitude and longitude.

Going in the other direction, there are several methods for converting an existing Skyfield position into latitude, longitude, and height:

<code>Geoid.latlon_of</code> (position)	Return the latitude and longitude of a <code>position</code> .
<code>Geoid.height_of</code> (position)	Return the height above the Earth's ellipsoid of a <code>position</code> .
<code>Geoid.geographic_position_of</code> (position)	Return the <code>GeographicPosition</code> of a <code>position</code> .
<code>Geoid.subpoint_of</code> (position)	Return the point on the ellipsoid directly below a <code>position</code> .

Once you have used either of the above approaches to build a `GeographicPosition`, it offers several methods:

`GeographicPosition.at` (t) At time `t`, compute the target's position

	relative to the center.
<code>GeographicPosition.lst_hours_at(t)</code>	Return the Local Apparent Sidereal Time, in hours, at time <code>t</code> .
<code>GeographicPosition.refract(altitude_degrees, ...)</code>	Predict how the atmosphere will refract a position.
<code>GeographicPosition.rotation_at(t)</code>	Compute rotation from GCRS to this location's altazimuth system.

Kepler orbits¶

See [Kepler Orbits](#) for computing the positions of comets, asteroids, and other minor planets.

Kepler orbit data¶

<code>load_mpcorb_dataframe(fobj)</code>	Parse a Minor Planet Center orbits file into a Pandas dataframe.
<code>load_comets_dataframe(fobj)</code>	Parse a Minor Planet Center comets file into a Pandas dataframe.
<code>load_comets_dataframe_slow(fobj)</code>	Parse a Minor Planet Center comets file into a Pandas dataframe.

Earth satellites¶

By downloading TLE satellite element sets, Skyfield users can build vector functions that predict their positions. See [Earth Satellites](#).

<code>EarthSatellite(line1, line2[, name, ts])</code>	An Earth satellite loaded from a TLE file and propagated with SGP4.
<code>EarthSatellite.from_omm(ts, element_dict)</code>	Build an EarthSatellite from OMM text fields.
<code>EarthSatellite.from_satrec(satrec, ts)</code>	Build an EarthSatellite from a raw sgp4 Satrec object.
<code>TEME</code>	The satellite-specific True Equator Mean Equinox frame of reference.

Stars and other distant objects¶

`Star` The position in the sky of a star or other fixed object.

Astronomical positions¶

The `ICRF` three-dimensional position vector serves as the base class for all of the following position classes. Each class represents an (x,y,z) `.position` and `.velocity` vector oriented to the axes of the International Celestial Reference System (ICRS), an inertial system that’s an update to J2000 and that does not rotate with respect to the universe.

<code>ICRF</code>	An (x,y,z) position and velocity oriented to the ICRF axes.
<code>Barycentric</code>	An (x,y,z) position measured from the Solar System barycenter.
<code>Astrometric</code>	An astrometric (x,y,z) position relative to a particular observer.
<code>Apparent</code>	An apparent (x,y,z) position relative to a particular observer.
<code>Geocentric</code>	An (x,y,z) position measured from the center of the Earth.

Positions are usually generated by the `at(t)` method of a vector function, rather than being constructed manually. But you can also build a position directly from a raw vector, or from right ascension and declination coordinates with `position_of_radec()`.

`position_of_radec`(ra_hours, dec_degrees[, ...]) Build a position object from a right ascension and declination.

All position objects offer five basic attributes:

<code>.position</code>	An (x,y,z) <code>Distance</code> .
<code>.velocity</code>	An (x,y,z) <code>Velocity</code> , or <code>None</code> .
<code>.t</code>	The <code>Time</code> of the position, or <code>None</code> .
<code>.center</code>	Body the vector is measured from.
<code>.target</code>	Body the vector is measured to.

All positions support these methods:

<code>ICRF.distance()</code>	Compute the distance from the origin to this position.
<code>ICRF.speed()</code>	Compute the magnitude of the velocity vector.
<code>ICRF.radec</code> ([epoch])	Compute equatorial RA, declination, and distance.
<code>ICRF.hadec()</code>	Compute hour angle, declination, and distance.
<code>ICRF.altaz</code> ([temperature_C, pressure_mbar])	Compute (alt, az, distance) relative to the observer’s horizon
<code>ICRF.from_altaz</code> ([alt, az, alt_degrees, ...])	Generate an Apparent position from an altitude and azimuth.

<code>ICRF.separation_from</code> (another_icrf)	Return the angle between this position and another.
<code>ICRF.frame_xyz</code> (frame)	Return this position as an (x,y,z) vector in a reference frame.
<code>ICRF.frame_xyz_and_velocity</code> (frame)	Return (x,y,z) position and velocity vectors in a reference frame.
<code>ICRF.frame_latlon</code> (frame)	Return longitude, latitude, and distance in the given frame.
<code>ICRF.frame_latlon_and_rates</code> (frame)	Return a reference frame longitude, latitude, range, and rates.
<code>ICRF.from_time_and_frame_vectors</code> (t, frame, ...)	Constructor: build a position from two vectors in a reference frame.
<code>ICRF.to_skycoord</code> ([unit])	Convert this distance to an AstroPy <code>SkyCoord</code> object.
<code>ICRF.phase_angle</code> (sun)	Return this position’s phase angle: the angle Sun-target-observer.
<code>ICRF.fraction_illuminated</code> (sun)	Return the fraction of the target’s disc that is illuminated.
<code>ICRF.is_sunlit</code> (ephemeris)	Return whether a position in Earth orbit is in sunlight.

In addition to the methods above, several subclasses of the base position class provide unique methods of their own:

<code>Barycentric.observe</code> (body)	Compute the <code>Astrometric</code> position of a body from this location.
<code>Astrometric.apparent</code> ()	Compute an <code>Apparent</code> position for this body.

Reference frames¶

<code>skyfield.framelib.true_equator_and_equinox_of_date</code>	The dynamical frame of Earth’s true equator and true equinox of date.
<code>skyfield.framelib.itrs</code>	The International Terrestrial Reference System (ITRS).
<code>skyfield.framelib.ecliptic_frame</code>	Reference frame of the true ecliptic and equinox of date.
<code>skyfield.framelib.ecliptic_J2000_frame</code>	Reference frame of the true ecliptic and equinox at J2000.
<code>skyfield.framelib.galactic_frame</code>	Galactic System II reference frame.

[skyfield.sgp4lib.TEME](#)

The satellite-specific True Equator Mean
Equinox frame of reference.

Constellations¶

`skyfield.api.load_constellation_map()`¶

Load Skyfield’s constellation boundaries and return a lookup function.

Skyfield carries an internal map of constellation boundaries that is optimized for quick position lookup. Call this function to load the map and return a function mapping position to constellation name.

```
>>> from skyfield.api import position_of_radec, load_constellation_map
>>> constellation_at = load_constellation_map()
>>> north_pole = position_of_radec(0, 90)
>>> constellation_at(north_pole)
'UMi'
```

If you pass an array of positions, you’ll receive an array of names.

`skyfield.api.load_constellation_names()`¶

Return a list of abbreviation-name tuples, like `('Aql', 'Aquila')`.

You can pass the list to Python’s `dict()` to build a dictionary that turns a constellation abbreviation into a full name:

```
>>> from skyfield.api import load_constellation_names
>>> d = dict(load_constellation_names())
>>> d['UMa']
'Ursa Major'
```

By swapping the order of the two items, you can map the other way, from a full name back to an abbreviation:

```
>>> f = dict(reversed(item) for item in load_constellation_names())
>>> f['Ursa Major']
'UMa'
```

`skyfield.data.stellarium.parse_constellations(lines)`¶

Return a list of constellation outlines.

Each constellation outline is a list of edges, each of which is drawn between a pair of specific stars:

```
[
```

```
(name, [(star1, star2), (star3, star4), ...]),
(name, [(star1, star2), (star3, star4), ...]),
...
]
```

Each name is a 3-letter constellation abbreviation; each star is an integer Hipparcos catalog number. See [Drawing a finder chart for comet NEOWISE](#) for an example of how to combine this data with the Hipparcos star catalog to draw constellation lines on a chart.

`skyfield.data.stellarium.parse_star_names(lines)`

Return the names in a Stellarium `star_names.fab` file.

Returns a list of named tuples, each of which offers a `.hip` attribute with a Hipparcos catalog number and a `.name` attribute with the star name. Do not depend on the tuple having only length two; additional fields may be added in the future.

Searching

`skyfield.searchlib.find_discrete()`

Find the times at which a discrete function of time changes value.

This routine is used to find instantaneous events like sunrise, transits, and the seasons. See [Searching for the dates of astronomical events](#) for how to use it yourself.

`skyfield.searchlib.find_maxima()`

Find the local maxima in the values returned by a function of time.

This routine is used to find events like highest altitude and maximum elongation. See [Searching for the dates of astronomical events](#) for how to use it yourself.

`skyfield.searchlib.find_minima()`

Find the local minima in the values returned by a function of time.

This routine is used to find events like minimum elongation. See [Searching for the dates of astronomical events](#) for how to use it yourself.

Osculating orbital elements

This routine returns osculating orbital elements for an object's instantaneous position and velocity.

`osculating_elements_of`(`position`[, ...]) Produce the osculating orbital elements for a position.

<code>OsculatingElements.apoapsis_distance</code>	Distance object
<code>OsculatingElements.argument_of_latitude</code>	Angle object
<code>OsculatingElements.argument_of_periapsis</code>	Angle object
<code>OsculatingElements.eccentric_anomaly</code>	Angle object
<code>OsculatingElements.eccentricity</code>	<code>numpy.ndarray</code>
<code>OsculatingElements.inclination</code>	Angle object
<code>OsculatingElements.longitude_of_ascending_node</code>	Angle object
<code>OsculatingElements.longitude_of_periapsis</code>	Angle object
<code>OsculatingElements.mean_anomaly</code>	Angle object
<code>OsculatingElements.mean_longitude</code>	Angle object
<code>OsculatingElements.mean_motion_per_day</code>	Angle object
<code>OsculatingElements.periapsis_distance</code>	Distance object
<code>OsculatingElements.periapsis_time</code>	Time object
<code>OsculatingElements.period_in_days</code>	<code>numpy.ndarray</code>
<code>OsculatingElements.semi_latus_rectum</code>	Distance object
<code>OsculatingElements.semi_major_axis</code>	Distance object
<code>OsculatingElements.semi_minor_axis</code>	Distance object
<code>OsculatingElements.time</code>	Time object
<code>OsculatingElements.true_anomaly</code>	Angle object
<code>OsculatingElements.true_longitude</code>	Angle object

Units ¶

<code>Distance</code>	Distance
<code>Velocity</code>	Velocity
<code>Angle</code>	Angle
<code>AngleRate</code>	Rate at which an angle is changing

All three kinds of quantity support one or more methods.

<code>Distance.au()</code>	Astronomical units.
<code>Distance.km()</code>	Kilometers (1,000 meters).
<code>Distance.m()</code>	Meters.

<code>Distance.length()</code>	Compute the length when this is an (x,y,z) vector.
<code>Distance.light_seconds()</code>	Return the length of this vector in light seconds.
<code>Distance.to(unit)</code>	Convert this distance to the given AstroPy unit.
<code>Velocity.au_per_d()</code>	Astronomical units per day.
<code>Velocity.km_per_s()</code>	Kilometers per second.
<code>Velocity.m_per_s()</code>	Meters per second.
<code>Velocity.to(unit)</code>	Convert this velocity to the given AstroPy unit.
<code>Angle.radians()</code>	Radians ($\tau = 2\pi$ in a circle).
<code>Angle.hours</code>	Hours (24^h in a circle).
<code>Angle.degrees</code>	Degrees (360° in a circle).
<code>Angle.arcminutes()</code>	Return the angle in arcminutes.
<code>Angle.arcseconds()</code>	Return the angle in arcseconds.
<code>Angle.mas()</code>	Return the angle in milliarcseconds.
<code>Angle.to(unit)</code>	Convert this angle to the given AstroPy unit.
<code>Angle.hms([warn])</code>	Convert to a tuple (hours, minutes, seconds).
<code>Angle.signed_hms([warn])</code>	Convert to a tuple (sign, hours, minutes, seconds).
<code>Angle.hstr([places, warn, format])</code>	Return a string like <code>12h 07m 30.00s</code> ; see Formatting angles .
<code>Angle.dms([warn])</code>	Convert to a tuple (degrees, minutes, seconds).
<code>Angle.signed_dms([warn])</code>	Convert to a tuple (sign, degrees, minutes, seconds).
<code>Angle.dstr([places, warn, format])</code>	Return a string like <code>181deg 52' 30.0"</code> ; see Formatting angles .
<code>AngleRate.radians</code>	Rate of change in radians.
<code>AngleRate.degrees</code>	Rate of change in degrees.
<code>AngleRate.arcminutes</code>	Rate of change in arcminutes.
<code>AngleRate.arcseconds</code>	Rate of change in arcseconds.
<code>AngleRate.mas</code>	Rate of change in milliarcseconds.
<code>Rate.per_day</code>	Units per day of Terrestrial Time.
<code>Rate.per_hour</code>	Units per hour of Terrestrial Time.
<code>Rate.per_minute</code>	Units per minute of Terrestrial Time.
<code>Rate.per_second</code>	Units per second of Terrestrial Time.

Trigonometry ¶

`position_angle_of(anglepair1, anglepair2)` Return the position angle of one position with

respect to another.

... • ...